Car Rental Application – Design Documentation

Project Overview

The Car Rental Application is a full-stack web application developed using:

- Backend: Spring Boot 3.5.8 with Java 21
- Frontend: React 19.2.6 with Vite 8.0.12
- Database: MySQL 8.0
- Build Tool: Apache Maven 3.9.x
- ORM Framework: Hibernate 6.x
- REST API Development: Spring Data JPA & Spring Web
- API Testing Tool: Postman 10.x
- Version Control: Git & GitHub
- IDEs Used: Spring Tool Suite 4 and Visual Studio Code 1.x

API Documentation: Swagger UI — Access the REST API documentation using the following link: http://localhost:8080/swagger-ui.html

The application allows users to:

- Reserve a vehicle
- Modify an existing reservation
- Cancel a reservation
- View available vehicle options with dynamically calculated pricing

The system supports the following vehicle categories:

- Sedan
- SUV
- Van
- Pickup Truck

The application is designed using clean architecture principles and object-oriented design practices to ensure maintainability, scalability, and extensibility.

2. Functional Requirements

The application exposes the following operations:

Operation	Description
Reserve Car	Create a new reservation
Modify Reservation	Update an existing reservation

Operation	Description
Cancel Reservation	Delete an existing reservation
Get Vehicle Options	Retrieve all vehicle types with calculated prices

3. Pricing Rules

Sedan

- If booking duration is less than 10 days:
    - Price = $20/day
- Otherwise:
    - Price = $15/day

Van

- Base price = $22/day
- Additional cleaning fee = 10% of total booking amount

SUV

- Base price = $15/day
- Additional charge = $0.50 per mile

Formula:

Total Amount =
(Number Of Days × 15)
+
(Daily Mileage × Number Of Days × 0.50)

Pickup Truck

- Base price = $30/day
- If customer has license experience less than 3 years:
    - Additional 10% surcharge applied

4. System Architecture

The application follows a layered architecture approach.

Backend Layers

Controller Layer
↓
Service Layer
↓

Repository Layer
↓
Database

5. Backend Design Decisions

Why Spring Boot?

Spring Boot was chosen because it provides:

- Rapid application development
- REST API support
- Dependency Injection
- Validation support
- Easy database integration
- Production-ready architecture

6. Use of Strategy Pattern

Problem

Each vehicle category has different pricing rules.

If all pricing logic were implemented using large if-else or switch statements, the code would become:

- Hard to maintain
- Difficult to extend
- Violating SOLID principles

Solution

To solve this, the Strategy Design Pattern was implemented.

Each vehicle category has its own pricing strategy class.

Pricing Strategy Interface

```
public interface PricingStrategy {

    VehicleCategory getCategory();

    double calculatePrice(BookingRequest request);
}
```

Strategy Implementations

Vehicle Type	Strategy Class
Sedan	SedanPricingStrategy
SUV	SuvPricingStrategy
Van	VanPricingStrategy
Pickup Truck	PickupTruckPricingStrategy

Benefits of Strategy Pattern

1. Open/Closed Principle

The system is open for extension but closed for modification.

New vehicle types can be added without changing existing code.

2. Separation of Concerns

Each pricing rule is isolated in its own class.

3. Maintainability

Pricing logic becomes easy to update and debug.

4. Scalability

Future pricing algorithms can be introduced easily.

7. Service Layer Design

The ReservationServiceImpl class contains the business logic.

Responsibilities include:

- Reserving vehicles
- Updating reservations
- Cancelling reservations
- Fetching available vehicle options
- Delegating pricing calculation to appropriate strategy classes

Dynamic Strategy Resolution

```
private PricingStrategy getStrategy(
      VehicleCategory category) {

   return pricingStrategies.stream()
         .filter(strategy ->
```

```
                strategy.getCategory() == category)
        .findFirst()
        .orElseThrow(() ->
                new RuntimeException(
                        "Invalid category"));
}
```

Why this approach?

This avoids hardcoded object creation and allows Spring Dependency Injection to manage all pricing strategies dynamically.

8. DTO Design

DTOs (Data Transfer Objects) were used to separate API contracts from database entities.

DTOs Used

DTO	Purpose
BookingRequest	Input for pricing calculation
ReservationRequest	Input for reservation APIs
ReservationResponse	Reservation API response
VehicleOptionResponse	Vehicle options response

Benefits of DTOs

- Prevent exposing database entities directly
- Better API security
- Cleaner request/response structures
- Easier frontend integration

9. Validation Design

Validation is implemented at both:

- Backend level
- Frontend level

This follows real-world enterprise application standards.

Backend Validation

Implemented using Jakarta Validation annotations.

Example:

```
@NotBlank(message = "Customer name is required")
private String customerName;

@Pattern(
   regexp = "^[A-Za-z ]+$",
   message = "Customer name must contain only alphabets"
)
private String customerName;

@Min(value = 1, message = "Days should be greater than 0")
private int numberOfDays;
```

Frontend Validation

React validation is added to improve user experience by preventing invalid API calls.

Examples:

- Prevent negative values
- Prevent numeric customer names
- Prevent empty fields

10. Repository Layer

Spring Data JPA Repository is used for database interaction.

Benefits:

- Reduced boilerplate code
- Automatic CRUD support
- Easy database integration

11. REST API Design

The application exposes RESTful APIs.

API Endpoints

Reserve Car

POST /api/reservations

Creates a new reservation.

Modify Reservation

PUT /api/reservations/{id}

Updates an existing reservation.

Cancel Reservation

DELETE /api/reservations/{id}

Deletes a reservation.

Get Vehicle Options

POST /api/reservations/options

Returns all available vehicle options sorted by total amount.

12. Sorting Logic

Vehicle options are returned in ascending order of total amount.

Implementation:

```
.sorted(Comparator.comparing(
    VehicleOptionResponse::getTotalAmount))
```

This improves customer experience by showing cheaper options first.

13. Frontend Design

The frontend was developed using ReactJS.

Frontend Features

- Responsive UI
- Modern glassmorphism design
- Dynamic form handling
- REST API integration using Axios
- Real-time validation
- Error handling
- Loading-friendly structure

14. Why React?

React was chosen because:

- Component-based architecture
- Fast rendering
- Reusable UI components
- Easy API integration
- Good scalability
- Industry-standard frontend framework

15. Error Handling

The application handles errors gracefully.

Backend Errors

Examples:

- Reservation not found
- Invalid input
- Validation failures

Custom exception:

ReservationNotFoundException

Frontend Errors

Displayed using alert messages:

- Failed reservation
- Validation issues
- API errors

16. Security & Validation Considerations

The application prevents:

- Negative booking values
- Invalid customer names
- Empty mandatory fields

Cross-Origin configuration:

@CrossOrigin(origins = "http://localhost:5173")

Used for secure frontend-backend communication during development.

17. Extensibility

The application was designed for future enhancements.

Adding New Vehicle Category

To add a new category:

Step 1

Add new enum value:

VehicleCategory.LUXURY

Step 2

Create new pricing strategy:

LuxuryPricingStrategy

No changes are required in existing business logic.

18. Object-Oriented Design Principles Used

SOLID Principles

Single Responsibility Principle

Each class has one responsibility.

Example:

- Pricing classes only calculate prices
- Controller only handles HTTP requests

Open/Closed Principle

New pricing strategies can be added without modifying existing logic.

Dependency Inversion Principle

Service depends on abstraction (PricingStrategy) instead of concrete classes.

19. Future Improvements

Possible future enhancements include:

- JWT Authentication
- Payment Gateway Integration
- Vehicle Availability Tracking

- Booking History
- Admin Dashboard
- Email Notifications
- Docker Deployment
- Unit Testing
- Integration Testing
- Swagger API Documentation

20. Conclusion

The Car Rental Application was designed using modern software engineering practices and object-oriented design principles.

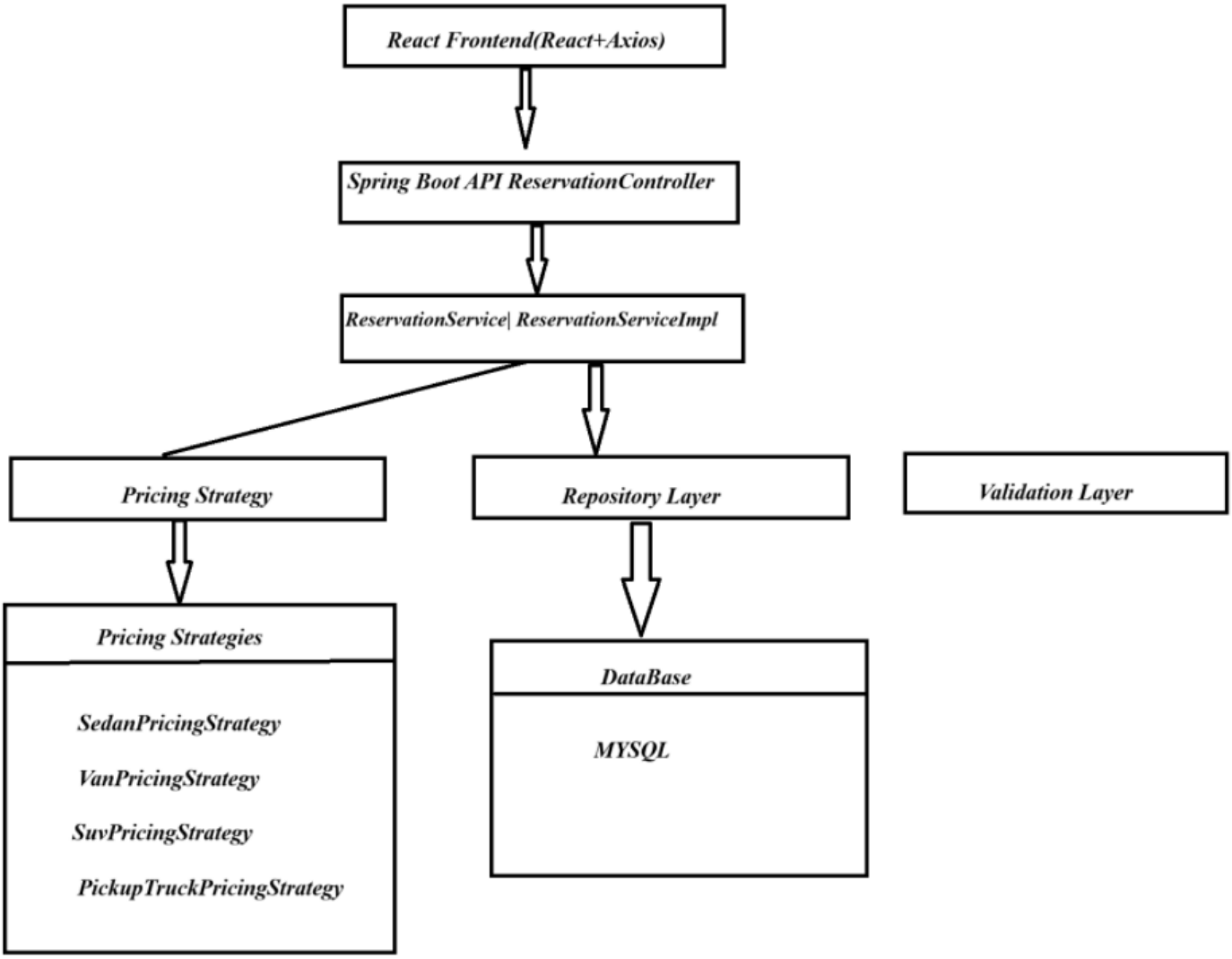
Key highlights:

- Clean layered architecture
- Strategy Pattern implementation
- Scalable and maintainable design
- Proper validation handling
- RESTful API architecture
- Responsive React frontend
- Extensible pricing engine

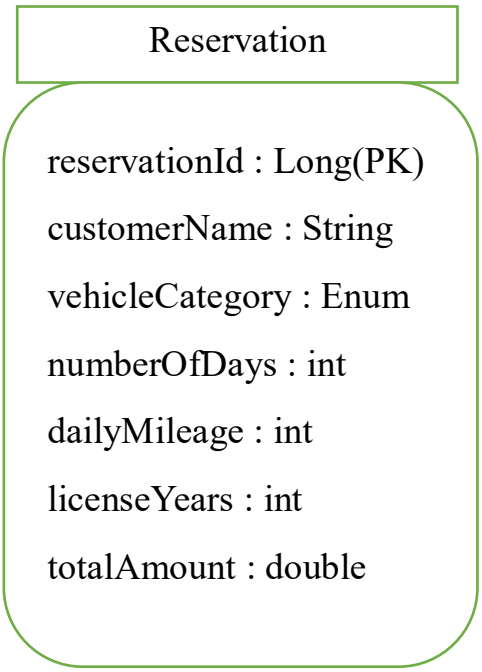
The system is production-oriented and designed to support future enhancements with minimal code modification.

UML Diagrams and Architecture Diagram

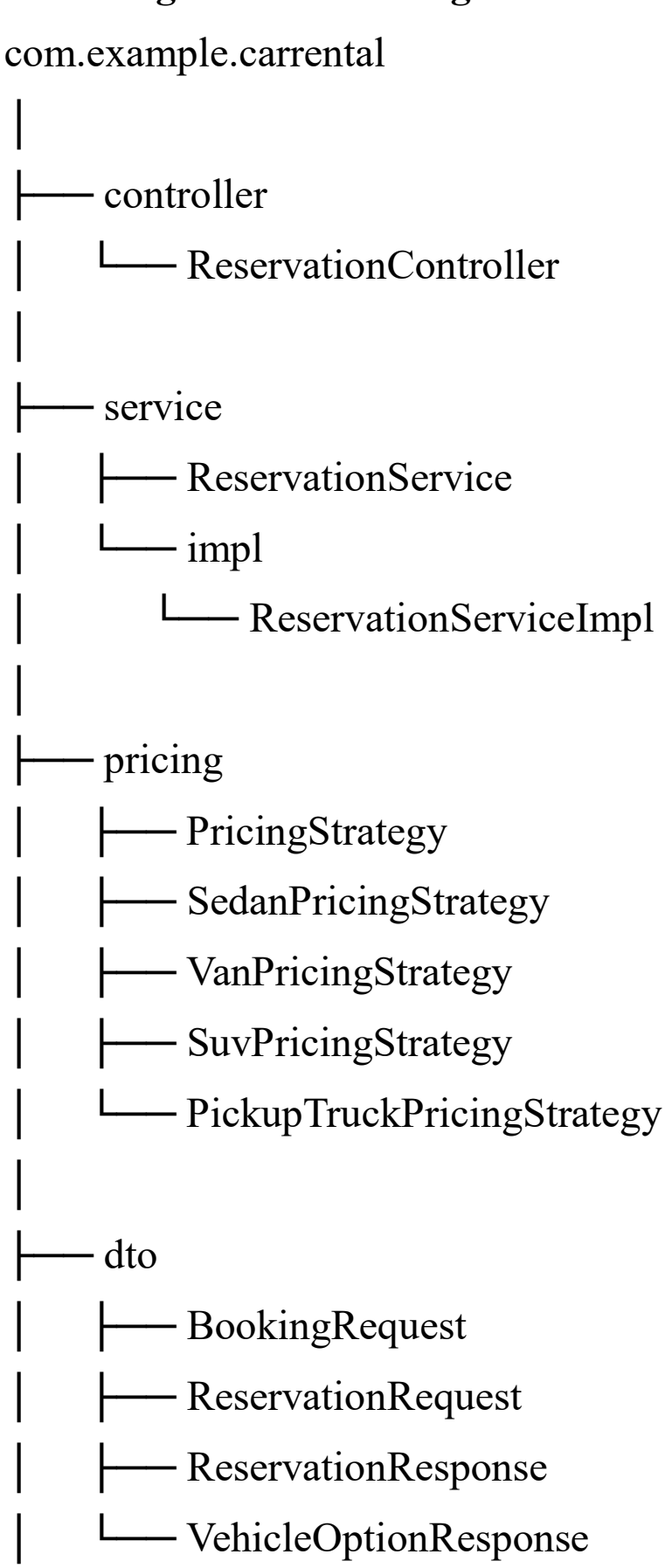
1. High-Level Architecture Diagram

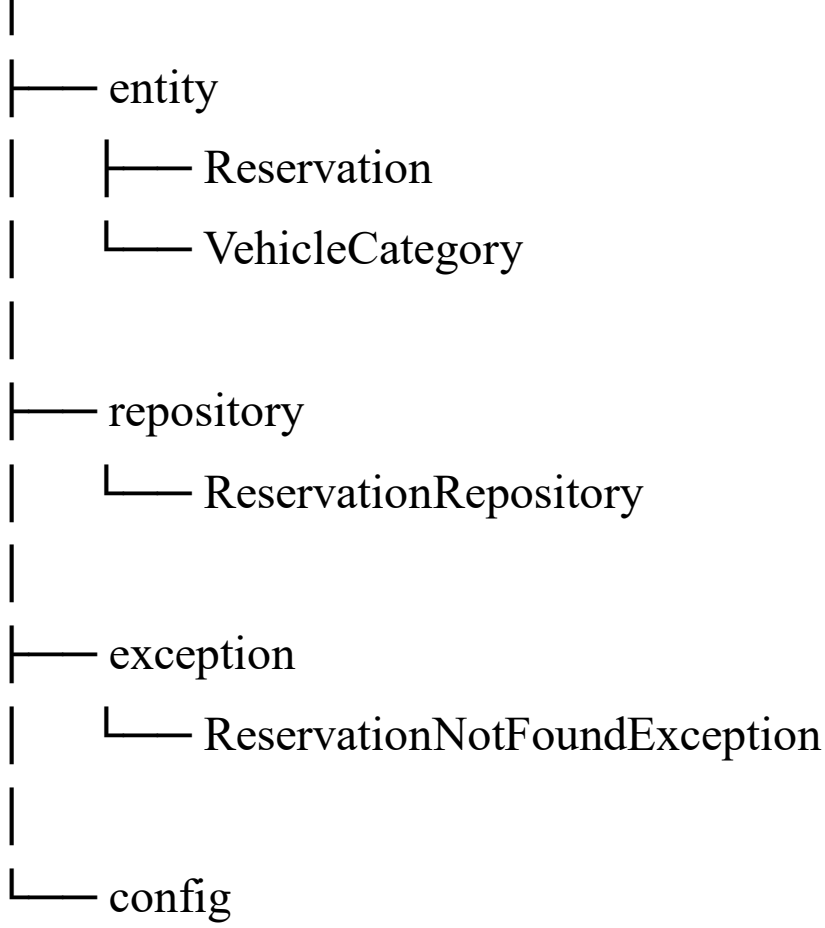


5. Entity Relationship Diagram (ER Diagram)



6. Package Structure Diagram





7. Why These Designs Were Chosen

Layered Architecture

The application follows a layered architecture to ensure clear separation of responsibilities across different parts of the system. Instead of placing all logic inside a single class, the application is divided into multiple layers, where each layer performs a specific role.

The main layers used in this application are:

- Controller Layer
- Service Layer
- Repository Layer
- Pricing Strategy Layer

Controller Layer

The Controller Layer is responsible for handling incoming HTTP requests from the frontend application.

Example responsibilities:

- Receiving API requests
- Validating request bodies
- Returning HTTP responses
- Mapping URLs to backend operations

In this project, the ReservationController handles operations such as:

- Reserving a car
- Modifying a reservation
- Cancelling a reservation
- Fetching vehicle options

The controller does not contain business logic. Instead, it delegates processing to the service layer.

Why this approach was chosen

Keeping controllers lightweight improves:

- Readability
- Maintainability
- Separation of concerns

This also makes debugging easier because request handling and business processing are separated.

Service Layer

The Service Layer contains the core business logic of the application.

In this project, the ReservationServiceImpl class performs:

- Reservation processing
- Vehicle pricing calculation coordination
- Reservation modification
- Reservation cancellation
- Vehicle option generation

The service layer acts as a bridge between:

- Controller Layer
- Repository Layer
- Pricing Strategy Layer

Why this approach was chosen

Business rules may become more complex in real-world systems. Keeping business logic inside services:

- Prevents code duplication
- Improves scalability
- Makes testing easier
- Keeps controllers clean

This design also supports future enhancements without affecting API endpoints.

Repository Layer

The Repository Layer is responsible for interacting with the database.

The ReservationRepository handles:

- Saving reservations
- Fetching reservations
- Updating reservations
- Deleting reservations

Spring Data JPA automatically provides CRUD operations, reducing boilerplate code.

Why this approach was chosen

Separating database operations from business logic provides:

- Cleaner architecture
- Better maintainability
- Easier migration to different databases
- Improved testing capability

The service layer does not directly interact with SQL queries, which improves abstraction.

Benefits of Layered Architecture

1. Separation of Concerns

Each layer has a clearly defined responsibility.

Example:

- Controllers manage APIs
- Services manage business rules
- Repositories manage data persistence

This prevents tightly coupled code.

2. Maintainability

If business rules change, modifications are usually limited to the service layer.

If database logic changes, only repository classes are affected.

This reduces the risk of breaking unrelated functionality.

3. Easier Testing

Each layer can be tested independently.

Examples:

- Controller testing
- Service unit testing
- Repository integration testing

This improves software quality.

4. Scalability

The application structure supports future expansion.

For example:

- Authentication module
- Payment module
- Notification module
- Admin dashboard

can be added easily without redesigning the application.

Strategy Pattern

The Strategy Design Pattern was implemented because each vehicle category follows different pricing rules.

For example:

- Sedan pricing depends on booking duration
- SUV pricing depends on mileage
- Van pricing includes cleaning charges
- Pickup Truck pricing depends on license experience

If all pricing logic were written inside a single method using multiple if-else conditions, the code would become:

- Difficult to read
- Hard to maintain
- Error-prone
- Difficult to extend

Problem Without Strategy Pattern

Without Strategy Pattern:

```
if(category == SEDAN) {
  // sedan pricing
}
else if(category == SUV) {
  // suv pricing
}
else if(category == VAN) {
  // van pricing
}
```

As the number of vehicle categories increases, the method becomes larger and more complex.

This violates:

- Open/Closed Principle
- Single Responsibility Principle

Solution Using Strategy Pattern

To solve this problem, a common interface called PricingStrategy was created.

Each vehicle category implements its own pricing logic independently.

Example implementations:

- SedanPricingStrategy
- VanPricingStrategy
- SuvPricingStrategy
- PickupTruckPricingStrategy

Each class is responsible only for its own pricing calculation.

Why Strategy Pattern Was Chosen

1. Extensibility

New vehicle categories can be added easily.

Example:

- Luxury Cars
- Electric Vehicles
- Sports Cars

To add a new category:

1. Add enum value

2. Create a new strategy class

No existing code needs modification.

This follows the Open/Closed Principle.

2. Avoids Large If-Else Chains

Each pricing algorithm is isolated into separate classes.

This improves:

- Code readability
- Maintainability
- Debugging

The service layer simply selects the appropriate strategy dynamically.

3. Better Maintainability

Pricing rules often change in real-world applications.

With Strategy Pattern:

- Each pricing rule can be updated independently
- Changes do not affect other vehicle categories

This reduces regression risk.

4. Better Reusability

Each pricing strategy can be reused independently in:

- Reports
- Billing systems
- Discount engines
- Promotions

5. SOLID Principles Compliance

The Strategy Pattern helps the application follow:

- Open/Closed Principle
- Single Responsibility Principle
- Dependency Inversion Principle

This results in cleaner enterprise-level architecture.

DTO Pattern

DTO stands for Data Transfer Object.

DTOs were used to transfer data between:

- Frontend and Backend
- Controller and Service layers

Examples used in this application:

- BookingRequest
- ReservationRequest
- ReservationResponse
- VehicleOptionResponse

Why DTO Pattern Was Chosen

Entities represent database structures, while DTOs represent API contracts.

Using entities directly in APIs is not recommended because:

- Sensitive fields may get exposed
- Database structure becomes tightly coupled with APIs
- Future database changes may break frontend contracts

DTOs solve these issues.

Benefits of DTO Pattern

1. Security

DTOs expose only required fields.

Example:
The frontend does not need access to internal database implementation details.

This improves application security.

2. Cleaner API Structure

DTOs provide:

- Well-structured request objects
- Well-structured response objects

This improves frontend integration and API readability.

3. Decoupling Between Layers

Changes in database entities do not directly impact frontend APIs.

For example:

- Internal entity fields can change
- Database normalization can change

without affecting API consumers.

4. Better Validation Support

Validation annotations are applied directly on DTOs.

Example:

```
@Min(1)
private int numberOfDays;
```

This keeps validation logic clean and centralized.

5. Easier Frontend Integration

Frontend developers can work with simplified request/response structures.

This improves:

- API usability
- Debugging
- Data consistency